CS2610: Computer Organization and Architecture Lab

Lab 6: Privilege Modes

9th March 2026

Objective

To investigate the RISC-V privileged architecture by implementing controlled transitions between Machine (M), Supervisor (S), and User (U) modes, and to configure exception delegation mechanisms.

Problem 1: The Privilege-Driven Dispatcher (100 Points)

In this assignment, you will implement a state driven dispatcher demonstrating transitions across the three primary RISC-V privilege levels. **The implementation must maintain a conditional execution flow** controlled by the value of register `a0`.
The execution begins at the **main** label in Machine mode. Initially, the program must shift directly from Machine → User mode, and then trap up to Supervisor mode. From this point, the Supervisor Trap Handler acts as a central dispatcher running in a continuous loop, while the Machine and User modes act as mathematical workers. This iterative approach allows for the continuous observation of privilege state modifications and Control and Status Register (CSR) behavior.

(a) Initial Downward Transition (M → U)

The program begins execution in **Machine mode**. Before normal execution starts, the system should transfer control to the **User mode** (ucode) where you will load two values, ***a1* and *a2***, from memory.
You will need to configure the **mstatus**, **mtvec** and the **mepc** register accordingly, and switch control using the **mret** instruction.

(b) Transition To Supervisor (U → S)

Move from ucode to the dispatcher (strap_handler). A system call should transfer control to a Supervisor-level trap handler. You might have to configure the **medeleg** register.

(c) The Dispatcher (Supervisor Trap Handler)

The Supervisor trap handler acts as a dispatcher that determines the next execution target.
A control value stored in **register a0** determines the execution flow:

- **Descend Condition (`a0` == 0):** Execution should proceed to a routine running in User mode - the **User Mode Worker (ucode1)**. This can be done by configuring the relevant control registers.
- **Ascend Condition (`a0` == 1):** The dispatcher must initiate an **upward privilege escalation** and shift the control to the **Machine mode Worker**. This is achieved by intentionally triggering a trap from within the Supervisor handler.

(d) The Workers and Return Paths

Depending on the dispatch direction, the target privilege mode will perform a specific arithmetic operation and return control to the dispatcher.

- **User Mode Worker (Addition):** While in User mode, perform the addition `a1 = a1 + a2`. Then, flip `a0` and issue an `ecall` to trap back to the dispatcher.
- **Machine Mode Worker (Multiplication):** Inside the `mtrap_handler`, perform the multiplication `a1 = a1 * a2`. Flip `a0` and then issue an `mret` back to the dispatcher.

Summary of Transition Mechanisms

Ensure your implementation utilizes the correct architectural instructions for each transition vector:

Vector	Transition Type	Primary Instruction
User → Supervisor	Upward (Call/Trap)	`ecall`
Supervisor → Machine	Upward (Call/Trap)	`ecall`
Machine → User/Supervisor	Downward (Return)	`mret`
Supervisor → User	Downward (Return)	`sret`

Template and Execution

Listing 1: Architectural Framework

```
.section .text
.global main

main:
    # Configure CSR registers (medeleg, mstatus, mepc, mtvec, stvec etc.)
    # Execute mret to initiate downward transition to User code
    mret

.align 4
mtrap_handler:
    # Process traps originating from the strap_handler
    # Perform the multiplication operation
    # Flip a0 to 0
    # Set mepc back to strap_handler and execute mret
    mret

scode:
    # Execute sret to initiate transition to User mode after setting the CSRs
    sret

.align 4
strap_handler:
    # The Dispatcher
    # Check if a0 == 0 or a0 == 1
    # If a0 == 0: jump to scode (to setup S to U transition)
    # If a0 == 1: execute ecall to jump up to mtrap_handler
    ecall

ucode:
    # Note: Handle initial variable loading here if using a single User block
```

```
    ecall

ucode1:
    # Perform the addition operation
    # Flip a0 to 1
    # The ecall should invoke the supervisor's trap handler
    ecall
```

Compilation and Execution

To compile your assembly program with the provided linker script and run it using the Spike simulator in debug mode, use the following commands:

```
# Compile the program
$ riscv64-unknown-elf-gcc -nostartfiles -T link.ld program.s

# Execute using Spike in debug mode, do not use the proxy kernel
$ spike -d program.out
```

Submission Instructions

1. This assignment must be completed on an individual basis.
2. Submit the commented assembly source file (`.s`).
3. Provide screenshots of the Spike debug console capturing the values of `mcause`, `mepc`, `mtvec` and `mstatus` in the mtrap_handler and values of `scause`, `sepc`, `stvec` and `sstatus` in the strap_handler.
4. Provide screenshots of the following instructions running in spike debug console. This is to verify privilege modes. They will raise an exception if the processor privilege mode is too low.

   ```
   # Under mtrap_handler
   csrr t6, mstatus

   # Under strap_handler
   csrr t6, sstatus
   ```

5. Provide screenshots of the ecall instruction at the end of ucode and the ecall instruction at the end of strap_handler while running in the spike debug console. It should show exception "trap_user_ecall" and exception "trap_supervisor_ecall" respectively. This is to verify privilege modes.